

Road to Ruby for A Linguistics Nerd

RubyKaigi 2025

Composed By: Hayato Ishida

For Japanese version, click [here](#)

Updated On: 15 May 2025

About Me

- Accounts
 - GitHub: [@hayat01sh1da](#)
 - X: [@hayat01sh1da](#)
 - LinkedIn: [@hayat01sh1da](#)
 - Speaker Deck: [@hayat01sh1da](#)
 - Docswell: [@hayat01sh1da](#)
- Occupation: Software Engineer
- Things I Am Into
 - Linguistics
 - Singing at Karaoke
 - Listening to Music
 - Watching Movies
 - Cat Feeding



Licences / Certifications

- English
 - TOEIC® Listening & Reading 915: Certified on December 2019
- Engineering
 - Information Security Management: Certified on November 2017
 - Applied Information Technology Engineer: Certified on June 2017
 - Fundamental Information Technology Engineer: Certified on November 2016
 - IT Passport: Certified on April 2016
- Others
 - Abacus 2nd Class: Certified on June 2002
 - Mental Arithmetic 3rd Class: Certified on February 2001

RubyKaigi 2025

Skill Stack

- Languages
 - Japanese: Native Proficiency
 - English: Full professional Proficiency
- Development
 - **Backend: Ruby(Ruby on Rails), Python**
 - Frontend: TypeScript + React.js, TypeScript +Vue.js
 - **Database: MySQL, PostgreSQL, MongoDB**
 - **Architecture: Monolith, Modular Monolith**
 - Hosting: AWS ESK
 - Versioning: Git, GitHub
 - **CI/CD: GitHub Actions, ArgoCD**
 - Monitoring: Datadog, Sentry

Work Histories

Period	Genre	Position	Jobs
Aug 2021 - Present	Saas(Educational Service)	Software Engineer	• Backend Development • Frontend Development • CI/CD Maintenance • Weekly Release Manager • Troubleshooting • Documentation • Technical Debt Resolution • Tech Blog Entries • LT Speaker
Feb 2020 - Dec 2020	SaaS(Chatbot Platform)	Backend Engineer	• Backend Development • Documentation • Technical Debt Resolution • Inspection of Alternative API

Work Histories

Period	Genre	Position	Jobs
Jun 2018 - Jan 2020	Contract-basis Developer	Software Engineer	• Backend Development • Frontend Development • Troubleshooting • Quality Assurance • Tech Blog Entries
Apr 2016 - Jan 2018	System Engineering Service	System Engineer	• Corporate Account Admin • Windows Server Maintenance • Security Admin Assistant • Internationalisation / Localisation

International-Exchange Activities

- Activities in University
 - English Linguistics Seminar(Focusing on Mass Media English)
 - International-Exchange Clubs(The 2nd Year)
 - [International-Exchange Programmes conducted by Japan Cabinet Office](#)(2013 - 2016)
 - Japanese Linguistics Course(The Final Year)
- Overseas Life Experience
 - Working Holiday in Australia(April 2014 - March 2015)
 - Language School for 1 month in Sydney
 - Work for 6 Months in [Hamilton Island Resort](#)
 - Volunteering for 1 Month as Assistant Teacher of Japanese Language at [St Ives High School](#) in NSW
- Other Activities
 - Keep Everyday Journal in English (April 2014 - Present)
 - [Sunrise Toastmasters Club](#)(February 2017 - March 2018)
 - [Vital Japan](#)(January 2018 - July 2019, October 2022 - February 2023)
 - Self Learning and Training of English Language
 - Participation Ruby-Related Tech Conferences

Prior to Lightning Talk...

The chief organiser of RubyKaigi, Mr. Akira Matsuda, allows me to use the official logo of RubyKaigi 2025 as the background image. No offence to its copyright.

Ref) https://x.com/a_matsuda/status/1918612709169672210



Agenda

1. [My Goal in RubyKaigi 2025](#)
2. [KaigiEffect -A Field I Have Been Into-](#)
3. [The Most Memorable Discussion Mr. Matz](#)
4. [As Is vs. To Be](#)
5. [References](#)

RubyKaigi 2025



1. My Goal in RubyKaigi 2025

RubyKaigi 2025

1. My Goal in RubyKaigi 2025

**To Find A Breakthrough where
I Can Contribute to Ruby
as A Linguistics Nerd.**

1. My Goal in RubyKaigi 2025

What brought me this goal?

- Have a long development experience in Ruby, but turned away from understanding of its complicated and profound lower layers
- The fact made me an immature developer who **can use Ruby but cannot make the most of Ruby**
- Strongly realised it was high time to seriously face and learn the ecosystem and lifecycle on lower layers of Ruby
- Wondered what I should cut my teeth on, so I decided to find something linguistic which may bring me a huge interest at RubyKaigi 2025

1. My Goal in RubyKaigi 2025

What strategies did I take?

- Picked up only the speeches focusing on the field I was interested in
- Took notes of keywords which did not make sense and/or attracted me, instead of trying to understand all
- Started learning the details of the keywords after RubyKaigi 2025 was over

RubyKaigi 2025

2. KaigiEffect -A Field I Have Been Into-

RubyKaigi 2025

2. KaigiEffect -A Field I Have Been Into-

A Big Interest in Language Processing Ecosystem and Lifecycle

- Used not to be interested in parser without any special reasons till this year
- [*Dissecting and Reconstructing Ruby Syntactic Structures*](#) presented by [@yдах](#) san brought me a huge interested in how Ruby source codes are interpreted

[< Back](#)



Yudai Takada
@yдах_
Ruby committer. A committer of [Lra](#)
[ma](#) LALR parser generator. Co-
Founder of [Kyobashi.rb](#).

JA

Dissecting and Reconstructing Ruby Syntactic Structures

The [parse.y](#) file, a blueprint for creating Ruby's parser, can be considered a direct representation of Ruby's grammatical structure. However, its syntax rules are highly complex, making it challenging to comprehend its architecture.

By dissecting and reconstructing the structure of the grammar rules, we can enhance the maintainability of parse.y file. Additionally, through this process, we will gain a deeper understanding of the underlying principles and patterns of Ruby's syntactic structure.

Let's embark on a journey to explore the internal workings of the Ruby language and uncover the true laws and principles that govern its grammar structure.

2. KaigiEffect -A Field I Have Been Into-

Discussion with My Colleagues and Collection of Materials

- Turned out the ecosystem was more complex than I knew
- Would love to know more about details of Ruby language processing system
- Discussed what materials are useful to learn Ruby language processing system works with my coworkers who are deeply familiar with the field
- Bookmarked the recommended websites and files
- I got recommendations on X as well(My best regard to all kind Rubyists)

* Having knowledgeable colleagues is 100% absolutely AN EMPLOYEE WELFARE

2. KaigiEffect -A Field I Have Been Into-

Discussion with My Colleagues and Collection of Materials

Thread

hayat01sh1da 休 24 Apr at 21:25
yda さんの Ruby Parser のセッションの後に @mopp さんに「低レイヤ理解するならこのリポジトリを教材に学習するといいですよ」と教えていただいたリンクを紛失した...
すみません、お手隙の際にもう一度ご共有いただけますでしょうか？ (GW 中にやろうと思っていた矢先でした) (edited)

108 replies

hayat01sh1da 休 24 Apr at 21:27
<https://github.com/rui314/9cc> でしたっけ？

rui314/9cc
A Small C Compiler
Stars 1866
Language C
Added by GitHub

mopp 24 Apr at 21:34
あ、それです
<https://www.sigbus.info/compilerbook> を見ながらやるといいですよ

1 1

ワイがやったやつはこちら
<https://github.com/mopp/9mm>
もう6年前で草も生えない

mopp/9mm
Hobby C Compiler
Stars 22
Language C
Added by GitHub

1 1

hayat01sh1da 休 24 Apr at 21:36
ありがとうございます、めっちゃ助かります！
mopp さんは草生やさなくてももう血が通ってるから笑

1 1

RubyKaigi 系の Bookmarks 整理しなきゃ

1 1

mopp 24 Apr at 22:02
コンピュータシステムの理論と実装でやるやつめっちゃ難に言うこれです
https://swkagami.hatenablog.com/entry/comput_02/mips
swk's log はてな別館
(講義動画) MIPSアーキテクチャ・swk's log はてな別館
前回に引き続き講義動画の公開です。MIPS 命令セットを題材として、アセンブリ言語レベルでのコンピュータの動作を学びます。
7 May 2020 (44 kB)

Hatena Blog
コンピュータアーキテクチャ
(講義動画) MIPSアーキテクチャ
swk's log はてな別館

2020-05-07 だからコロナ禍で YouTube 動画化されたんだな？

hayat01sh1da 休 24 Apr at 22:04
ちなみになんですが、「[低レイヤを知りたい人のためのCコンパイラ作成入門]」を参考書、mmtk-ruby を問題集に手を動かすのと、明快入門 コンパイラ・インタプリタ開発『C処理系を作りながら学ぶ 林端比古実用マスターシリーズ』のとどどちらが学習コストがいいですかね？

https://quipper.slack.com/archives/C04NT6AAGJX/p1745499787697829?thread_ts=1745497534.637939&cid=C04NT6AAGJX
無料公開しちゃうのすごい太っ腹...

mopp 2020-05-07 だからコロナ禍で YouTube 動画化されたんだな？
From a thread in rubykaigi-ja 24 Apr View reply

mopp 24 Apr at 22:05
mmtk の Ruby binding 公式であったのか
一番学びたいのはどの部分でしょうか？
mmtk はメモリ管理の分野のうちの GC 特化の部分なので、言語処理系の話からはだいぶ離れますね

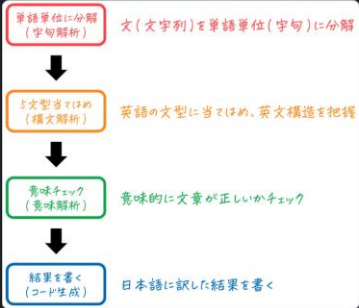
hayat01sh1da 休 24 Apr at 22:06
一番学びたいのは構文解析ですね、優先度 S です！
その周辺知識も必要であれば学びたいという感じです。
GC のエコシステムやライフサイクルに明るくないのめかなりコンプレックスなので学びたいですが、優先度 B-A くらいです。 (edited)
Peter さんのプレゼンも理解できるようになりたいです、さっさと行きたいけど絶対難しいことやってますよね (edited)

2. KaigiEffect -A Field I Have Been Into-

Discussion with My Colleagues and Collection of Materials

mopp 24 Apr at 22:09
今適当に探したんですが、まずこのあたりの概要を理解するとよさそうです
<https://www.momoyama-usagi.com/entry/info-calc-sys11>

■ 工業大学生ももやまのうさぎ塾
うさぎでもわかる計算機システム Part11 コンパイラの処理の流れ（字句解析と意味解析のしくみ）
こんにちは、ももやまです。今回はコンパイラの処理の流れについて、字句解析と意味解析を中心にまとめていきたいと思います。前提としてオートマトンや文脈自由文法の知識があると理解が早まるかと思います。(136 kB)



単語単位に分割 (字句解析) 文(文字列)を単語単位(単語)に分割
↓
文型当てはめ (構文解析) 英語の文型に当てはめ、英文構造を把握
↓
意味チェック (意味解析) 意味的に文章が正しいかチェック
↓
結果を書く (コード生成) 日本語に訳した結果を書く

1 1

広告が多いけど普通にわかりやすいな

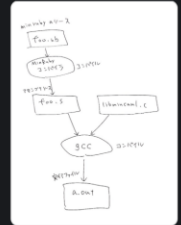
hayat01sh1da 24 Apr at 22:11
導入が私にとってめっちゃ親近感あります！
オートマトンよく聞くなーって思ってたらかこの周辺の用語だったんですね
自然言語処理独学した時に目にしたのか👀

mopp 24 Apr at 22:12
あー、どうなのでしょう
言語処理系に应用されることが多いですが
オートマトンは「計算とかなにか？」の表現技法の一つが始まりだった気がします

mopp 24 Apr at 22:20
ああ、RubyでつくるRubyがありましたね
構文解析を学びたいなら RubyでつくるRuby だけで十分読めるかもしれません
アセンブラからは逃げられない様子です
<https://blog.agile.esm.co.jp/entry/minruby-compiler>

ESM アジャイル事業部 開発者ブログ
Rubyでつくる、ミニでRubyなコンパイラ - ESM アジャイル事業部 開発者ブログ
こんパイラ〜（挨拶）、電子の海に漂うはかなき泡沫（うたかた）、はたけやまです。みなさん、書籍「RubyでつくるRuby」をご存知ですか？Rubyを使ってRubyのサブセット「MinRuby」のインタプリタをすることで言語処理系作成のエッセンスを学ぼう！という本です。RubyでつくるRuby
<https://www.lambdanote.com/products/ruby-ruby> 今回は、この本のMinRubyを題材に、簡易なMinRubyコンパイラをRubyで作成してみようと思います。（この記事はESM Advent Calendar 2023 の4日目の記事になります）Gitリボ...

4 Dec 2023



hayat01sh1da 24 Apr at 22:23
おお、そんなに強力な書籍だったとは(さすが遠藤さん)
アセンブラからは逃げられない様子です
もう諦めました、立ち向かいますw

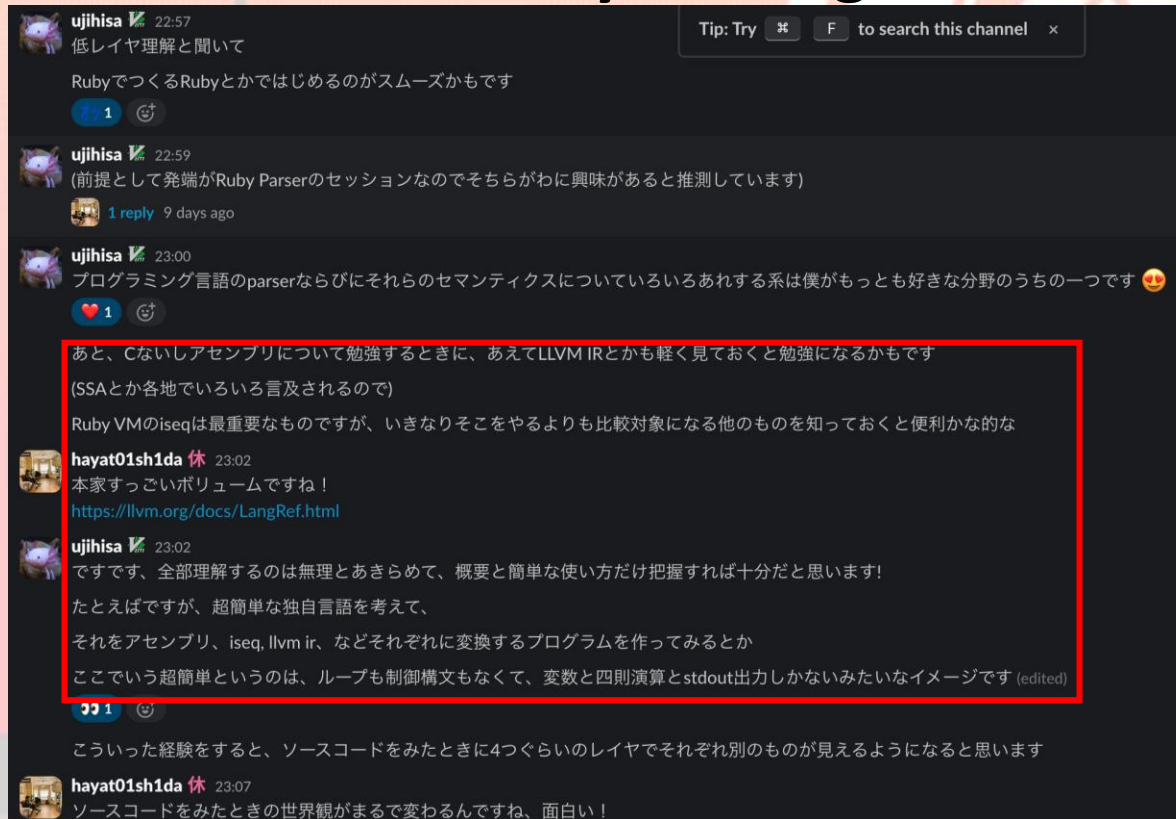
mopp 24 Apr at 22:23
これやるなら3はいらないかもですね
目次を見る限りだと9ccのがやりこたえはあるかもです

1 1

C言語はねえ
Rubyを書いた後の身には文字列処理がアホほどめんどくさいんですよ

2. KaigiEffect -A Field I Have Been Into-

Discussion with My Colleagues and Collection of Materials



The screenshot shows a Discord chat interface with a dark theme. At the top, there's a search bar with the text "Tip: Try # F to search this channel". The chat history shows several messages from a user named "ujihisa". The first message is about understanding low-level Ruby. The second message is about Ruby Parser sessions. The third message is about programming language parsers and semantics. The fourth message, which is highlighted with a red box, discusses LLVM IR and Ruby VM's iseq. The fifth message is from "hayat01sh1da" linking to an LLVM document. The sixth message is from "ujihisa" explaining the complexity of understanding LLVM IR. The seventh message is from "hayat01sh1da" about the world view changing when looking at source code.

ujihisa 22:57
低レイヤ理解と聞いて
RubyでつくるRubyとかでは始めるのがスムーズかもです
1

ujihisa 22:59
(前提として発端がRuby Parserのセッションなのでそちらがわに興味があると推測しています)
1 reply 9 days ago

ujihisa 23:00
プログラミング言語のparserならびにそれらのセマンティクスについていろいろあれする系は僕がもっとも好きな分野のうちの一つです 🙄
1

あと、Cないしアセンブリについて勉強するときに、あえてLLVM IRとかも軽く見ておくと勉強になるかもです
(SSAとか各地でいろいろ言及されるので)
Ruby VMのiseqは重要なものですが、いきなりそこをやるよりも比較対象になる他のものを知っておくと便利かな的な

hayat01sh1da 23:02
本家すっごいボリュームですね！
<https://llvm.org/docs/LangRef.html>

ujihisa 23:02
です、全部理解するのは無理とあきらめて、概要と簡単な使い方だけ把握すれば十分だと思います！
たとえばですが、超簡単な独自言語を考えて、
それをアセンブリ、iseq、llvm ir、などそれぞれに変換するプログラムを作ってみるとか
ここでいう超簡単というのは、ループも制御構文もなく、変数と四則演算とstdout出力しかないみたいなイメージです (edited)
1

こういった経験をする、ソースコードをみたときに4つぐらいのレイヤでそれぞれ別のものが見えるようになると思います

hayat01sh1da 23:07
ソースコードをみたときの世界観がまるで変わるんですね、面白い！

2. KaigiEffect -A Field I Have Been Into-

Discussion with My Colleagues and Collection of Materials

ujihisa 23:07

- A. プログラムの持つ「意味」をみたレイヤ (人間の脳が認識している世界)
- B. ソースコードそのまま全体を文字列としてみたレイヤ (人間の目が認識している世界)
- C. トークン列として見たレイヤ (parserが見ている世界、またはテキストエディタのシンタックスハイライト機構などが見ている世界)
- D. ASTとして見たレイヤ (狭義のcompilerが見ている世界)
- E. 内部の命令列として見たレイヤ (runtimeが見ている世界)

(edited)

書いてたら5つになっちゃった

hayat01sh1da 休 23:10

2つめが私(石田その①)、1・3つ目が TomPng さん(石田その②w)ってイメージを持ちました(TomPng さんは全部分かった上でトークン列に関心をおいていると思いますが)

内部の命令列として見たレイヤ (runtimeが見ている世界)

今年の Class#new 高速化でアーロンさんがお話しされてた記憶があります！

1

ujihisa 23:12

たとえばMatzさんが言語のデザインについて考えているときはAとBで、その互換性について考えるときや実装するときは主にCとDで (edited)

1

Aaronさんがobject生成まわりについて考えている (必然的にGCまわりについても考えている)ときは実質Eのみ、みたいな感じですね!

1

2. KaigiEffect -A Field I Have Been Into-

What am I learning by?

- [LLVM Language Reference Manual](#)
- [Parsing Techniques](#)
- [低レイヤを知りたい人のための C コンパイラ作成入門](#)
- [MIPS アーキテクチャ](#)
- ✓ [うさぎでもわかる計算機システム Part10 コンパイラの基礎・分割コンパイルの流れ](#)
- ✓ [うさぎでもわかる計算機システム Part11 コンパイラの処理の流れ\(字句解析と意味解析のしくみ\)](#)
- ✓ [Ruby Parser 開発日誌\(14\) - LR parser 完全に理解した](#)
- ✓ [オートマトンと言語理論 学習記 #1: オートマトンとは何か -](#)
- ✓ [オートマトンと言語理論 学習記 #2: 有限オートマトン -](#)
- ✓ [Ruby の中身の予備知識 RubyKaigi の前に知り鯛！](#)
- ✓ [たのしい Ruby の構文解析ツアー](#)
- ✓ [LL 法と LR 法の違いは？調べてみた！ -完全版-/Comparing LL and LR parse algorithm -EX Edition-](#)

2. KaigiEffect -A Field I Have Been Into-

What am I delivering by?

- ✓ [Ruby でつくる、ミニで Ruby なコンパイラ](#)
- [9cc](#)
- [lr-parser-101](#)
- [『Ruby でつくる Ruby ゼロから学びなおすプログラミング言語入門』](#)

RubyKaigi 2025

2. KaigiEffect -A Field I Have Been Into-

Participation in A Brand-New Ruby Community

[Shikumi.rb](https://shikumi.rb)

organised by [@hashimoto-kei](https://twitter.com/hashimoto-kei) and [@utsubo](https://twitter.com/utsubo)

- Born on May 2025 and still a new-born baby Ruby community
- Sessions mainly held online
- Focuses on language processing in Ruby, which is the very lifesaver for me.
- Perfect attendance so far(as of 09 May 2025)

3. The Most Memorable Discussion Mr. Matz

RubyKaigi 2025

3. The Most Memorable Discussion Mr. Matz

I knew Mr. Matz was so interested in Linguistics that he shared my advent calendar entry on Twitter(Current. X) as mentioned in [*Life and Death -Natural Languages vs. Programming Languages-*](#).

Also, I myself am sure that Ruby is highly conscious of as smooth a writing experience as English mentioned in [*Recommendation of Comparative Linguistics - Learn Identity of Programming Language from Ruby vs. Python-*](#).



3. The Most Memorable Discussion Mr. Matz

Highly fortunately, I was on the same table as Mr. Matz had a seat on at [Code Party RubyKaigi 2025](#) organised by ANDPAD and succeeded in having the following discussion.

* H = hayat01sh1da, M = Mr. Matz

H) Mr. Matz, you created Ruby learning from Perl, so it has some similarities to that. One of them is [Linguistic and Cognitive Considerations](#). **Did you have such a linguistic and cognitive philosophy** when you created, grew and released Ruby?

M) Mr. Larry Wall, creator of Perl, has a backbone as a linguistics while I majored in Computer Science. Our backgrounds are different from those of each other, but I **really value** [Linguistic and Cognitive Considerations](#) **taking care of compatibilities.**

3. The Most Memorable Discussion Mr. Matz

H) As a Rubyist and someone who majored English linguistics, I believe that Ruby provides a smooth writing experience as English. I feel like writing **Ruby is quite similar to writing an English poem**. I do understand your philosophy and my hypophysis mentioned [Recommendation of Comparative Linguistics -Learn Identity of Programming Language from Ruby vs. Python-](#) is proved to be correct.

M) **I think highly of not just how method works but what the method works for.** For example, let's think about `Enumerable#collect` and `Enumerable#map`. Both of them returns a new array with the results of data processing of every element of the receiver, inside of the block. The former does it for data collection, whereas the latter for data mapping. As you see, this example demonstrates different methods of the same data processing have as many different purposes.

H) It reminds me what you mentioned in [107. The Importance of Naming on Japanese Edition of 97 Things Every Programmer Should Know](#) as naming vividly reflects the design.

3. The Most Memorable Discussion Mr. Matz

M) *In terms of naming, I made an error in the past. I thought about naming for a method which takes its receiver as the block argument of returns the result of processing it. At first, I named it ``Object#yield_self``. It makes sense in order to describe how it works, but I failed to illustrate what it is for. Later on, I created its alias as ``Object#then``. Data processing is done once prior to its receiver, and 'then' it focuses on what is done. **It makes more sense not just in terms of the way the method works but the purpose the method works for.***

H) *Totally make sense. To compare it to English syntax, most of methods treat its receiver as the subject, themselves as the predicate verb, its argument(s) as the object(s) and inside of block as the subordinate clause(e.g. noun clause, adjective clause or adverb clause). **However, you adopt an adverb 'then' as a method, which strikes me as incredibly impressive!***

3. The Most Memorable Discussion Mr. Matz

Works as a phrasal verb

```
1  [10.downto(1)].yield self [{ it.sort }]
2  s [S] [V1] V3 O [S] [V1]
3  => [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4  10.downto(1).then { it.sort }
5  s V1 O (and) s V1
6  => [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

- S = Subject
- V = Verb
 - V₁: SV
 - V₂: SVC
 - V₃: SVO
 - V₄: SVO₁O₂
 - V₅: SVOC
- O = Object
 - O₁: Indirect Object
 - O₂: Direct Object
- C = Complement
- [Subordinate Clause]
- (Modifier)

* [Enumerable#sort](#) returns a new array as the return value by [rb_ary_sort_bang](#) function through [enum_sort](#) function, so a Range object is implicitly converted to an Array object.

3. The Most Memorable Discussion Mr. Matz

Object#yield_self vs. Object#then

- **Object#yield_self**

- Actually, yields its receiver as the following block argument
- Does not describe how it is handled within the block by its name
- The block works as a subordinate noun clause and has SV₁ in it, which complex sentence with nested structure brings about higher cognitive load

- **Object#then**

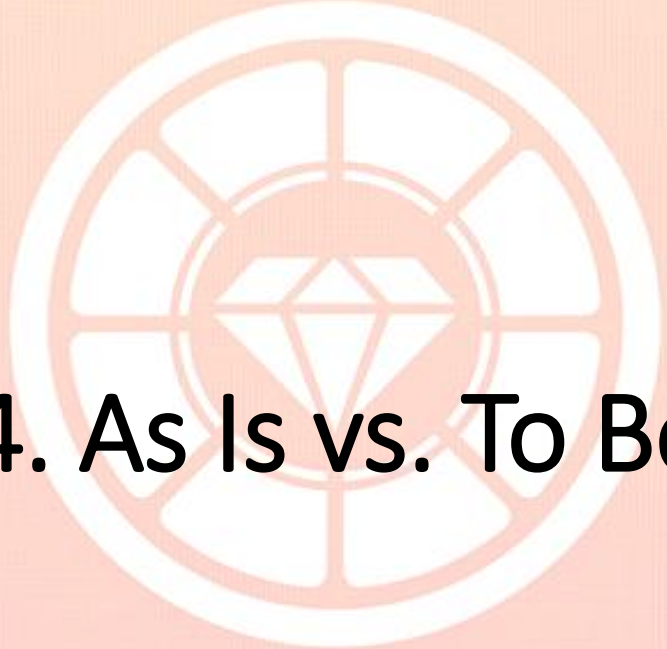
- Literally works as *and then* (Coordinate Conjunction + Adverb)
- Focuses on how the corresponding pronoun *it* is handled within the block
- That is why, succeeds in describing how *it*, the referrer of the first principal clause, is handled within the block by its name
- The block works as another principal clause, which simple sentence without nested structure provides better readability

3. The Most Memorable Discussion Mr. Matz

This discussion with Mr. Matz lasted only for 10 minutes or more, but I was honoured to have such a fruitful time.

I cannot be more grateful to Mr. Matz as a big fan of Ruby and linguistics.





4. As Is vs. To Be

RubyKaigi 2025

4. As Is vs. To Be

AsIs

- A Rubyist who implements web applications and scripts in Ruby

ToBe

- A Rubyist who knows **ecosystems and lifecycles of lower layers of Ruby** and optimise performance in web applications and scripts I implement
- A Rubyist who understands **how Ruby language processing works and commits enhancements**
- A Rubyist who **enriches and optimises linguistic and cognitive design of Ruby**



5. References

RubyKaigi 2025

5. References

- [RubyKaigi 2025 > Schedule](#)
 - Last Accessed On: 05 May 2025
- [LLVM Language Reference Manual](#)
 - Last Accessed On: 05 May 2025
- [PARCING TECHNIQUES -A Practical Guide-](#)
 - Last Accessed On: 05 May 2025
- [低レイヤを知りたい人のためのCコンパイラ作成入門](#)
 - Last Accessed On: 05 May 2025
- [MIPS アーキテクチャ](#)
 - Last Accessed On: 05 May 2025

5. References

- [うさぎでもわかる計算機システム Part10 コンパイラの基礎・分割コンパイルの流れ](#)
 - Last Accessed On: 05 May 2025
- [うさぎでもわかる計算機システム Part11 コンパイラの処理の流れ\(字句解析と意味解析のしくみ\)](#)
 - Last Accessed On: 05 May 2025
- [Ruby Parser 開発日誌\(14\) - LR parser 完全に理解した](#)
 - Last Accessed On: 05 May 2025
- [オートマトンと言語理論 学習記 #1: オートマトンとは何か -](#)
 - Last Accessed On: 05 May 2025
- [オートマトンと言語理論 学習記 #2: 有限オートマトン -](#)
 - Last Accessed On: 05 May 2025

5. References

- [Ruby の中身の予備知識 RubyKaigi の前に知り鯛！](#)
 - Last Accessed On: 05 May 2025
- [たのしい Ruby の構文解析ツアー](#)
 - Last Accessed On: 05 May 2025
- [LL 法と LR 法の違いは？調べてみた！-完全版-/Comparing LL and LR parse algorithm -EX Edition-](#)
 - Last Accessed On: 05 May 2025
- [9cc](#)
 - Last Accessed On: 05 May 2025
- [lr-parser-101](#)
 - Last Accessed On: 05 May 2025

5. References

- Yusuke Endo, [Ruby でつくる Ruby ゼロから学びなおすプログラミング言語入門](#), Tokyo, 2017
- [APIdoc > Ruby](#)
 - Last Accessed On: 05 May 2025
- [Perl 6 Design Philosophy](#)
 - Last Accessed On: 05 May 2025
- [97 Things Every Programmer Should Know](#)
 - Last Accessed On: 05 May 2025
- [プログラマが知るべき97のこと](#)
 - Last Accessed On: 05 May 2025



EOD

RubyKaigi 2025